Report - Version 3: Training Foundations

Introduction

Version 3 reorganizes the NumPy neural character model from Version 2 into a structured training experiment.

The corpus, vocabulary, adjacent-character task, numerical representation, one-hot inputs, 27 by 27 weight matrix, softmax probabilities, cross-entropy objective, and autoregressive generator remain unchanged. The new concept is the training process: reproducible data splitting, shuffled mini-batches, and systematic validation monitoring.

```
Characters
↓
Integer identifiers
↓
Train / validation split
↓
Shuffled mini-batches
↓
Gradient updates
↓
Validation monitoring
↓
Generated text
```

Goal of Version 3

The model should be able to:

- reuse the corpus, vocabulary, numerical encoding, and neural bigram architecture from Version 2;
- divide the 439 adjacent input-target examples into reproducible training and validation sets;
- shuffle only the training examples at the beginning of every epoch;
- update the weights with mini-batch gradient descent;
- measure training and validation loss after every epoch;
- identify the epoch with the lowest validation loss without using validation examples for weight updates;
- reproduce initialization, data splitting, training, and text generation from explicit local random seeds.

Architecture

The complete Version 3 flow is:

```
Corpus: 440 characters
↓
Vocabulary: 27 characters
↓
439 adjacent input-target pairs
↓
One-hot input matrix: (439, 27)
↓
Reproducible 80 / 20 example split
↓
Training: 351 | Validation: 88
↓
Shuffled mini-batch updates
↓
Train and validation loss monitoring
↓
Autoregressive generator
```

The context still contains one character. Version 3 therefore remains the same neural character bigram model introduced in Version 2.

Training and validation split

A local NumPy generator with seed 42 creates one random permutation of the 439 example indices. The first 80 percent are assigned to training and the remaining examples to validation.

```
split_position = int(439 * 0.8) = 351
training examples = 351
validation examples = 88
```

The two subsets are disjoint by example index, and together they contain all 439 examples. Validation examples are used only to measure loss and never to update the weights.

Because the split operates on individual adjacent-character examples, repeated bigram types can occur in both subsets. The validation set therefore holds out example positions, not necessarily previously unseen character transitions.

Unchanged numerical model

Every input character is still represented by a one-hot vector of length 27. The model still contains one trainable 27 by 27 weight matrix.

```
Input matrix shape: (439, 27)
Weight matrix shape: (27, 27)
Trainable parameters: 729
```

Initial weights are small random values generated from a local NumPy seed of 42, preserving the same transparent parameterization used in Version 2.

Mini-batches and shuffling

At the beginning of each training epoch, the 351 training examples are shuffled with a dedicated local random generator. The shuffled examples are then processed in mini-batches of size 32.

```
351 training examples
↓
10 full mini-batches of 32
+
1 final mini-batch of 31
↓
11 weight updates per epoch
```

With 100 training epochs, the model performs 1,100 mini-batch parameter updates. The validation set is not shuffled for learning because it never participates in optimization.

Systematic reproducibility

The same seed value, 42, is used by separate local random generators for weight initialization, data splitting, training shuffles, and the generation demonstration. These generators are independent objects, so the notebook does not depend on NumPy's global random state.

Training objective

For each mini-batch, the model applies the same calculations introduced in Version 2:

- multiply one-hot inputs by the weight matrix to obtain logits;
- apply numerically stable softmax to obtain next-character probabilities;
- measure cross-entropy against the correct target IDs;
- calculate logits and weight gradients directly in NumPy;
- update the weights with gradient descent.

```
logits_gradient = probabilities - targets
logits_gradient /= batch_size
weights_gradient = batch_inputs.T @ logits_gradient
weights -= learning_rate * weights_gradient
```

Training configuration:

```
Learning rate: 1.0
Batch size: 32
Epochs: 100
Seed: 42
```

Training result

```
Initial train loss: 3.297216
Final train loss: 1.987639
Initial validation loss: 3.297493
Final validation loss: 2.903679
Best validation loss: 2.893800
Best validation epoch: 64
```

Training loss decreases throughout the run. Validation loss initially decreases, reaches its minimum at epoch 64, and then rises slightly. This is expected because only the training loss is directly optimized.

Version 3 records the best validation epoch for analysis, but it does not implement early stopping or restore a checkpoint. The final trained weights are the weights after epoch 100.

Learned probabilities

After training, one row of the final weight matrix can still be interpreted as a next-character distribution for a chosen context.

For the character `m`, the highest probabilities are:

```
o: 0.2497 e: 0.1818 a: 0.1810
s: 0.1126 p: 0.1120 space: 0.0427
```

These values are learned only from the training subset, so they need not match the full-corpus empirical proportions or the Version 2 full-batch result.

Neural text generation

Generation preserves the same autoregressive loop used in Version 2:

- convert the current character into its integer ID;
- select the corresponding trained logits and apply softmax;
- sample the next ID from the probability distribution;
- convert the sampled ID back into a character;
- use that character as the next context.

The demonstration generates 300 characters from the final epoch-100 weights. A local seed makes the result reproducible.

Included checks

The notebook assertions verify example counts, one-hot and parameter shapes, complete non-overlapping data splits, loss-history lengths, finite losses, decreasing training loss, normalized probabilities, deterministic retraining, reproducible generation, and requested output length.

A complete top-to-bottom run ends with the message `All checks passed.`

What Version 3 teaches

The central progression is:

```
split -> shuffle -> batch -> update -> validate -> reproduce -> sample
```

Version 3 separates model architecture from training procedure. The predictive model is intentionally unchanged so that the educational focus stays on how experiments are organized and evaluated.

Interpretation

The training set updates the parameters; the validation set measures the same next-character objective on held-out example positions. Monitoring both losses makes the gap between optimization and validation behavior visible.

The minimum validation loss at epoch 64 is an observed diagnostic result, not a selected checkpoint. Text generation still uses the final epoch-100 parameters.

Limitations

- the context contains one character only;
- the corpus is small and embedded in the notebook;
- the random example-level split can place repeated character transitions in both training and validation;
- there is no early stopping, checkpoint restoration, hidden layer, embedding, automatic differentiation, or framework;
- validation monitors held-out examples but is not a robust estimate of generalization to a different corpus;
- generated text preserves local character patterns but has no long-term coherence.

Conclusion

Version 3 turns the NumPy neural character model into a reproducible training experiment with a train/validation split, mini-batches, epoch-level monitoring, and deterministic reruns. It prepares the project for Version 4, where the same small model can be rebuilt with TensorFlow and automatic differentiation after the underlying calculations are already understood.